Prefix Sum Technique in Java

Cumulative totals, range queries, and subarray problems

Core idea: Prefix sum stores cumulative totals so that the sum of a range can be calculated without adding every element in that range again. After building the prefix array once, each range-sum query can be answered in O(1) time.

1. What is prefix sum?

Suppose an array contains [2, 4, 1, 5, 3]. A prefix sum is the running total from the beginning of the array up to each position. The total through index 2 is 2 + 4 + 1 = 7.

Prefix sums are useful for repeated sums of continuous ranges, cumulative totals, and subarrays satisfying a sum-related condition. Instead of recalculating every range, we reuse the cumulative information.

Two common definitions

Inclusive prefix: prefix[i] stores the sum from index 0 through i. For [2, 4, 1, 5, 3], it is [2, 6, 7, 12, 15].

Prefix with a leading zero: prefix[0] = 0 and prefix[i + 1] stores the sum of the first i + 1 elements. The same input gives [0, 2, 6, 7, 12, 15]. This version simplifies range calculations.

Building the prefix array

```
int[] arr = {2, 4, 1, 5, 3};
long[] prefix = new long[arr.length + 1];

for (int i = 0; i < arr.length; i++) {
    prefix[i + 1] = prefix[i] + arr[i];
}
// prefix = [0, 2, 6, 7, 12, 15]
```

Each element is processed once: building the prefix array takes O(n) time and O(n) extra space.

2. Finding the sum of any range

With a leading-zero prefix array, the sum from index left to right (both inclusive) is:

```
rangeSum(left, right) = prefix[right + 1] – prefix[left]
```

prefix[right + 1] contains the total from index 0 through right. Subtracting prefix[left] removes all elements before left, leaving only the requested range.

Example

For arr = [2, 4, 1, 5, 3], prefix = [0, 2, 6, 7, 12, 15]. The sum from index 1 to 3 is prefix[4] – prefix[1] = 12 – 2 = 10. The range [4, 1, 5] also sums to 10.

Index	0	1	2	3	4
Array value	2	4	1	5	3
Prefix values*	2	6	7	12	15

*The prefix row above aligns each array index with prefix[index + 1].

Reusable Java methods

```
static long[] buildPrefix(int[] arr) {
    long[] prefix = new long[arr.length + 1];
    for (int i = 0; i < arr.length; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }
    return prefix;
}

static long rangeSum(long[] prefix, int left, int right) {
    if (left < 0 || right < left || right + 1 >= prefix.length) {
        throw new IllegalArgumentException("Invalid range");
    }
    return prefix[right + 1] - prefix[left];
}
```

Complexity and limitation

Building takes O(n); each query takes O(1). For q queries, total time is O(n + q), compared with O(nq) in the worst case if each range is scanned. If array values change frequently, the prefix array becomes outdated; a Fenwick tree or segment tree may be more appropriate.

3. Problem 1 — Range sum queries

Problem: Given an integer array and queries [left, right], return the sum of each inclusive range.

Input: arr = [2, 4, 1, 5, 3], queries = [[1, 3], [0, 2], [2, 4]]
Expected results: 10, 7, 9

Step-by-step

1. Build prefix = [0, 2, 6, 7, 12, 15].
2. Query [1, 3]: prefix[4] – prefix[1] = 12 – 2 = 10.
3. Query [0, 2]: prefix[3] – prefix[0] = 7 – 0 = 7.
4. Query [2, 4]: prefix[5] – prefix[2] = 15 – 6 = 9.

Complete Java program

```
public class RangeSumQueries {
    static long[] buildPrefix(int[] arr) {
        long[] prefix = new long[arr.length + 1];
        for (int i = 0; i < arr.length; i++) {
            prefix[i + 1] = prefix[i] + arr[i];
        }
        return prefix;
    }

    static long rangeSum(long[] prefix, int left, int right) {
        return prefix[right + 1] - prefix[left];
    }

    public static void main(String[] args) {
        int[] arr = {2, 4, 1, 5, 3};
        int[][] queries = {{1, 3}, {0, 2}, {2, 4}};
        long[] prefix = buildPrefix(arr);
        for (int[] q : queries) {
            System.out.println(rangeSum(prefix, q[0], q[1]));
        }
    }
}
```

Output: 10, then 7, then 9.

Interview point: preprocessing costs O(n), each query costs O(1), and extra space is O(n). This is ideal when many queries are asked on an array that does not change.

4. Problem 2 — Count subarrays with sum k

Problem: Given an integer array that may contain positive, zero, or negative values, count contiguous subarrays whose sum equals k.

Example: nums = [1, 1, 1], k = 2. Output is 2: the ranges [0, 1] and [1, 2] both sum to 2.

A prefix array can sum a known range, but this problem asks us to find and count matching ranges. Use a running prefix sum plus a HashMap that records how often each earlier prefix sum has appeared.

Key equation

Let currentPrefix be the sum from index 0 through the current position. If an earlier prefix equals currentPrefix – k, the elements after that earlier prefix through the current position sum to k.

At each element, add the frequency of (currentPrefix – k) to the answer, then record currentPrefix. Initialise the map with prefix sum 0 having frequency 1, to count valid subarrays starting at index 0.

Complete Java program

```
import java.util.HashMap;
import java.util.Map;

public class SubarraySumK {
    static int countSubarrays(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        freq.put(0, 1);

        int prefixSum = 0;
        int count = 0;

        for (int value : nums) {
            prefixSum += value;
            count += freq.getOrDefault(prefixSum - k, 0);
            freq.put(prefixSum, freq.getOrDefault(prefixSum, 0) + 1);
        }
        return count;
    }

    public static void main(String[] args) {
        int[] nums = {1, 1, 1};
        System.out.println(countSubarrays(nums, 2));
    }
}
```

Output: 2

5. Trace, complexity, and revision

Trace for nums = [1, 1, 1], k = 2

Value	Prefix sum	Needed prefix	Count added	Total
1	1	−1	0	0
1	2	0	1	1
1	3	1	1	2

At the second element, currentPrefix = 2, so the needed earlier prefix is 0. It has appeared once; one valid subarray ends here. At the third element, currentPrefix = 3, so the needed prefix is 1. It has appeared once; another valid subarray ends here. Total = 2.

Why count before updating the map?

First look for earlier prefix sums that make a valid range ending at the current index. Then store the current prefix for future positions. This order also matters when k = 0, because the current prefix should not count itself as an earlier prefix.

Complexity

Expected time is O(n), since each element performs a constant number of HashMap operations. Space is O(n) in the worst case. Use long for prefix sums and counts if the constraints can exceed the int range.

Common mistakes

• With a leading-zero prefix array, use prefix[right + 1] – prefix[left].
• Do not forget freq.put(0, 1).
• Count matches before recording the current prefix.
• The HashMap approach works with negative values and zeros; a simple sum-based sliding window generally does not.
• Consider long if sums or the number of subarrays can become large.

Practice questions

1. Answer many range-sum queries for an unchanged array.
2. Check whether an array contains a subarray with sum 0.
3. Count subarrays with sum k when negative values are allowed.
4. In a binary array, count subarrays with equal numbers of 0s and 1s by transforming 0s into −1s.

Key takeaway: Prefix sum turns repeated range addition into subtraction of cumulative totals. Use a prefix array for fast range queries. Use running prefix sums with a frequency map when counting subarrays with a target sum.